Regresión Logística en Python

Explicación detallada del notebook

Clasificación binaria para control de calidad

Alberto Muñoz

24 de marzo de 2026

¿Qué problema resuelve la regresión logística?

La regresión logística se utiliza cuando queremos predecir una **clase** y no un valor continuo.

Por ejemplo:

- ¿Un correo es **spam** o **no spam**?
- ¿Un cliente **comprará** o **no comprará**?
- ¿Una pieza es **defectuosa** o **aceptable**?
- ¿Un paciente tiene **alto riesgo** o **bajo riesgo**?

La idea central es modelar la **probabilidad** de pertenecer a una clase:

$$P(y = 1 \mid x)$$

y después convertir esa probabilidad en una decisión.

¿Por qué no usar regresión lineal?

Supongamos que queremos clasificar entre dos clases: $y \in \{0, 1\}$.

Si usamos regresión lineal:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

aparecen varios problemas:

- La salida puede ser menor que 0 o mayor que 1
- Eso no tiene sentido como probabilidad
- La relación entre variables y probabilidad rara vez es lineal en todo el rango

Necesitamos una función que:

1. reciba una combinación lineal de variables
2. la transforme a un valor entre 0 y 1

Esa función es la **sigmoide**.

La idea geométrica

La regresión logística construye primero un puntaje lineal:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Ese puntaje puede tomar cualquier valor real:

$$z \in (-\infty, +\infty)$$

Luego, transforma ese puntaje a una probabilidad mediante:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

De esta manera:

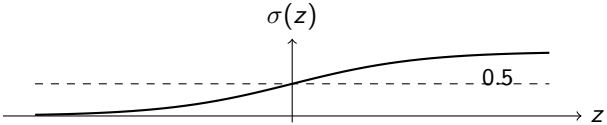
$$0 < \sigma(z) < 1$$

Interpretación:

- Si z es muy grande, la probabilidad se acerca a 1
- Si z es muy negativa, la probabilidad se acerca a 0
- Si $z = 0$, entonces la probabilidad es 0.5

La función sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Propiedades importantes:

- Es continua y suave
- Comprime cualquier valor real al intervalo $(0, 1)$
- Permite interpretar la salida como probabilidad

Formulación del modelo

En regresión logística modelamos:

$$P(y = 1 \mid x) = \sigma(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)$$

o de forma compacta:

$$P(y = 1 \mid x) = \sigma(\beta^T x)$$

donde:

- x es el vector de entrada
- β es el vector de parámetros
- $\sigma(\cdot)$ es la función sigmoide

La predicción final suele obtenerse con un umbral:

$$\hat{y} = \begin{cases} 1, & \text{si } P(y = 1 \mid x) \geq 0.5 \\ 0, & \text{si } P(y = 1 \mid x) < 0.5 \end{cases}$$

Interpretación con odds y log-odds

La regresión logística no es lineal en la probabilidad, pero sí en el **logit**.

$$p = P(y = 1 \mid x)$$

$$\text{odds} = \frac{p}{1-p}$$

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

Esto significa que el modelo es lineal en el **logaritmo de las odds**.

Interpretación de β_j:

- Si $\beta_j > 0$, al aumentar x_j, aumentan las odds de la clase 1
- Si $\beta_j < 0$, al aumentar x_j, disminuyen las odds
- e^{β_j} representa el factor multiplicativo sobre las odds

¿Cómo se entrenan los parámetros?

Los parámetros β no se obtienen con mínimos cuadrados como en regresión lineal.

Se estiman maximizando la probabilidad de observar los datos:

$$\mathcal{L}(\beta) = \prod_{i=1}^{m} p_i^{y_i}(1 - p_i)^{(1-y_i)}$$

donde:

$$p_i = \sigma(\beta^T x_i)$$

En la práctica se usa la log-verosimilitud:

$$\log \mathcal{L}(\beta) = \sum_{i=1}^{m} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Y normalmente se minimiza su versión negativa, conocida como **cross-entropy loss** o **log loss**.

Función de costo

La función de costo para clasificación binaria es:

$$J(\beta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right]$$

donde:

$$p_i = \sigma(\beta^T x_i)$$

Interpretación:

- Si el modelo asigna alta probabilidad a la clase correcta, el costo disminuye
- Si asigna alta probabilidad a la clase incorrecta, el costo crece mucho

Esto hace que la regresión logística penalice con fuerza las predicciones muy seguras pero equivocadas.

Optimización

Como no hay una solución cerrada simple, los parámetros se ajustan con métodos iterativos como:

- Gradient Descent
- Stochastic Gradient Descent
- Newton-Raphson
- LBFGS

La idea es actualizar:

$$\beta \leftarrow \beta - \eta \nabla J(\beta)$$

donde:

- η es la tasa de aprendizaje
- $\nabla J(\beta)$ es el gradiente de la función de costo

En bibliotecas como `scikit-learn`, este proceso ya viene implementado.

Frontera de decisión

Para clasificación binaria, el modelo divide el espacio de características con una frontera de decisión.

Si:

$$P(y = 1 \mid x) = 0.5$$

entonces:

$$\beta^T x = 0$$

Esto significa que la frontera es lineal en el espacio de variables:

- una recta en 2D
- un plano en 3D
- un hiperplano en dimensiones mayores

Por eso la regresión logística es un clasificador lineal, aunque su salida sea probabilística.

El papel del umbral

Aunque muchas veces usamos 0.5 como umbral, este puede cambiarse según la aplicación.

$$\hat{y} = \begin{cases} 1, & \text{si } p \geq \tau \\ 0, & \text{si } p < \tau \end{cases}$$

donde τ es el umbral.

Ejemplos:

- En medicina, puede preferirse un umbral bajo para no dejar pasar casos peligrosos
- En fraude, quizá conviene subir el umbral para evitar falsas alarmas

Cambiar el umbral modifica el equilibrio entre:

- sensibilidad
- especificidad
- falsos positivos
- falsos negativos

Métricas de evaluación

No basta con reportar accuracy. En clasificación, especialmente con clases desbalanceadas, conviene revisar varias métricas.

Métrica	¿Qué mide?
Accuracy	Proporción total de aciertos
Precision	Qué proporción de positivos predichos sí eran positivos
Recall	Qué proporción de positivos reales fueron detectados
F1-score	Balance entre precision y recall
ROC-AUC	Capacidad de separar clases en distintos umbrales

La mejor métrica depende del costo del error en el problema real.

Matriz de confusión

La matriz de confusión resume las decisiones del clasificador:

	Predice 0	Predice 1
Real 0	Verdadero Negativo	Falso Positivo
Real 1	Falso Negativo	Verdadero Positivo

A partir de esta matriz pueden calcularse:

$$\text{Precision} = \frac{VP}{VP + FP} \qquad \text{Recall} = \frac{VP}{VP + FN}$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Esta tabla ayuda a entender no solo cuánto falla el modelo, sino **cómo** falla.

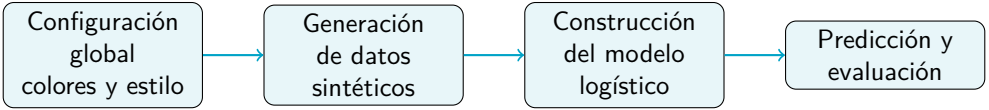
Veamos el código propuesto

- Explicar línea por línea la lógica del notebook de regresión logística.
- Mostrar cómo se construye un ejemplo sintético de **clasificación binaria**.
- Relacionar el código con los conceptos clave: datos, modelo, entrenamiento, probabilidades, umbral y métricas.
- Reservar un espacio claro para **4 gráficas** que se agregarán después como imágenes.

Idea central

El notebook simula un problema de **control de calidad de lavadoras**. A partir de variables como vibración y temperatura, el modelo estima la probabilidad de que una unidad pertenezca a la clase positiva.

Mapa general del código



Qué hace realmente el notebook

1) define un estilo visual, 2) crea datos con ruido, 3) entrena una regresión logística con descenso por gradiente, 4) calcula probabilidades y 5) resume el desempeño con métricas.

Bloque 1 - Configuración global

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import Ellipse, FancyBboxPatch
from matplotlib.colors import ListedColormap
import warnings; warnings.filterwarnings('ignore')
```

- `numpy` permite trabajar con vectores, matrices y generación de números aleatorios.
- `matplotlib.pyplot` se usa para producir las figuras que ilustran el problema.
- `gridspec`, `Ellipse` y `FancyBboxPatch` ayudan a maquetar visualizaciones más ricas.
- `ListedColormap` construye paletas personalizadas para clases.
- `warnings.filterwarnings('ignore')` evita ruido visual en la salida del notebook.

Bloque 2 - Identidad visual y parámetros de estilo

```
MABE_BLUE, MABE_DARK  = '#00A3C7', '#003B4A'
MABE_LIGHT, MABE_ORANGE = '#E8F6F9', '#E07C24'
MABE_GREEN, MABE_GRAY = '#2E8B57', '#6E6E6E'
MABE_RED = '#C0392B'
C3 = [MABE_BLUE, MABE_ORANGE, MABE_GREEN]
C4 = [MABE_BLUE, MABE_ORANGE, MABE_GREEN, MABE_RED]
CMAP2 = ListedColormap([MABE_BLUE, MABE_ORANGE])
```

- Se define una **paleta corporativa** consistente.
- C3 y C4 son listas listas para colorear categorías.
- CMAP2 crea un mapa de color de **dos clases**, útil en problemas binarios.

Lectura conceptual

No cambia el modelo matemático, pero sí mejora la claridad de la comunicación visual.

Bloque 3 - Ajustes de Matplotlib y reproducibilidad

```
plt.rcParams.update({'font.family':'sans-serif','font.size':9,
    'axes.titlesize':11,'axes.titleweight':'bold','axes.labelsize':9,
    'axes.spines.top':False,'axes.spines.right':False,
    'figure.facecolor':'white','figure.dpi':120})
np.random.seed(42)
```

- `rcParams.update(...)` fija reglas de presentación para todas las gráficas.
- Se eliminan los ejes superiores y derechos para lograr un estilo más limpio.
- `figure.dpi` controla la nitidez de la imagen exportada.
- `np.random.seed(42)` asegura que los datos aleatorios sean **reproducibles**. Cada vez que se corre el notebook, el experimento sintético arranca igual.

Bloque 4 - Función auxiliar para títulos

```python
def stitle(fig, t, s=''):
    fig.suptitle(t, fontsize=13, fontweight='bold', color=MABE_DARK, y=0.98)
    if s:
        fig.text(0.5, 0.94, s, ha='center', fontsize=9, color=MABE_GRAY)
```

Qué hace esta función

Centraliza el estilo de los títulos de las figuras: el texto principal aparece arriba con formato fuerte y, si se proporciona `s`, agrega un subtítulo centrado.

- `fig.suptitle(...)` define el título general de la figura.
- `fig.text(...)` inserta texto en coordenadas relativas de la figura.
- Esta función mejora la **consistencia visual** entre las 4 gráficas.

Bloque 5 - Inicio del ejemplo de regresión logística

```
from scipy.special import expit
n = 300
vib = np.random.normal(0,1,n)
tmp = np.random.normal(0,1,n)
```

- `expit` es la función sigmoide: transforma cualquier valor real en una probabilidad entre 0 y 1.
- `n = 300` fija el tamaño del conjunto de datos sintético.
- `vib` representa una variable de vibración.
- `tmp` representa una variable de temperatura.

Interpretación

Cada observación corresponde a una posible lavadora inspeccionada. El modelo verá dos señales numéricas por ejemplo: vibración y temperatura.

Bloque 6 - Construcción de la etiqueta binaria

```
y = (0.8*vib + 0.6*tmp + np.random.normal(0,0.3,n) > 0.5).astype(int)
```

- Se crea una combinación lineal de las variables: $0.8\,vib + 0.6\,tmp$.
- Se añade **ruido gaussiano** con `np.random.normal(0,0.3,n)` para evitar una frontera demasiado perfecta.
- Se compara contra el umbral 0.5.
- `astype(int)` convierte `True/False` en `1/0`.

Idea matemática

$$y_i = \mathbb{1}(0.8\,vib_i + 0.6\,tmp_i + \epsilon_i > 0$$

Bloque 7 - Matriz de entrada y término de sesgo

```
X = np.column_stack([vib, tmp])
X_b = np.column_stack([np.ones(n), X])
w = np.zeros(3)
```

- `X` reúne las dos variables predictoras en una matriz de tamaño 300×2.
- `X_b` agrega una primera columna de unos para incorporar el **bias** o intercepto.
- `w = np.zeros(3)` inicializa tres parámetros:

$$w = [w_0, w_1, w_2]$$

donde w_0 es el sesgo y w_1, w_2 ponderan vibración y temperatura.

Bloque 8 - Entrenamiento con descenso por gradiente

```python
for _ in range(200):
    p = expit(X_b @ w)
    w -= 0.1*(X_b.T @ (p - y))/n
```

Qué ocurre en cada iteración

1. Se calcula X_b@w, es decir, la combinación lineal actual.
2. Se aplica `expit` para obtener probabilidades estimadas p.
3. Se mide el error como $(p - y)$.
4. Se actualizan los pesos en dirección opuesta al gradiente.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \, \frac{X_b^T (p - y)}{n}, \qquad \eta = 0.1$$

Interpretación matemática del entrenamiento

- La regresión logística no predice una clase de forma directa; predice una **probabilidad**.
- La sigmoide comprime la salida lineal a $[0, 1]$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Cuando z crece, la probabilidad se acerca a 1.
- Cuando z es muy negativo, la probabilidad se acerca a 0.

En lenguaje simple

El modelo aprende qué tan importante es cada variable para mover la predicción hacia **aprobado** o **rechazado**.

Bloque 9 - Probabilidades y clases predichas

```
probs = expit(X_b @ w)
yp = (probs > .5).astype(int)
```

- `probs` contiene la probabilidad estimada para cada observación.
- El umbral de decisión es 0.5.
- Si `probs > 0.5`, la predicción final es clase 1; de lo contrario, clase 0.

Detalle importante

Separar **probabilidad** de **clase** es crucial. La probabilidad da información de confianza; la clase es la decisión final.

Bloque 10 - Conteo de la matriz de confusión

```
tp=np.sum((yp==1)&(y==1)); fp=np.sum((yp==1)&(y==0))
fn=np.sum((yp==0)&(y==1)); tn=np.sum((yp==0)&(y==0))
```

- `tp`: verdaderos positivos.
- `fp`: falsos positivos.
- `fn`: falsos negativos.
- `tn`: verdaderos negativos.

	Real 1	Real 0
Pred 1	TP	FP
Pred 0	FN	TN

Estas cuatro cantidades condensan el comportamiento del clasificador.

Bloque 11 - Métricas de desempeño

```
prec=tp/(tp+fp)
rec=tp/(tp+fn)
f1=2*prec*rec/(prec+rec)
acc=(tp+tn)/n
```

Interpretación de cada métrica

- **Precisión**: de todo lo que el modelo marcó como positivo, cuánto era correcto.
- **Recall**: de todos los positivos reales, cuántos detectó el modelo.
- **F1**: equilibrio entre precisión y recall.
- **Accuracy**: proporción total de aciertos.

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}, \quad \text{Accuracy} = \frac{TP + TN}{n}$$

Bloque 12 - Preparación de la figura compuesta

```
fig, ax = plt.subplots(2, 2, figsize=(13, 7.5))
```

- Se crea una figura con **4 subgráficas**, organizadas en una malla 2×2.
- `figsize=(13, 7.5)` aproxima una relación panorámica ideal para presentación.
- Este bloque sugiere que el notebook está pensado para contar una historia visual en cuatro partes.

Nota para la versión final

Aquí es donde tus 4 gráficas entran de manera natural. Abajo te dejo 4 slides reservados para insertarlas como imágenes, cada uno con texto de apoyo y referencia explícita.

Lectura integral del notebook

1. Se crea un entorno visual limpio y reproducible.
2. Se simula un fenómeno industrial con dos variables y ruido.
3. Se construye una etiqueta binaria a partir de una regla oculta.
4. Se entrena un clasificador logístico desde cero.
5. Se convierten probabilidades en decisiones.
6. Se evalúa el desempeño con métricas estándar.
7. Se prepara una narrativa visual con 4 gráficas.

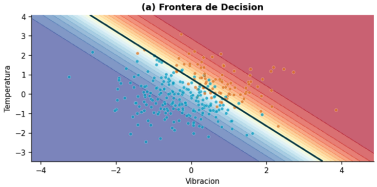
Mensaje pedagógico

Este notebook es excelente para explicar que **Machine Learning** no empieza en bibliotecas complejas: con álgebra lineal, probabilidad y unas cuantas líneas de Python ya se puede construir un clasificador completo.

Gráfica 1 - Distribución de los datos y clases

- Esta imagen debe mostrar cómo se distribuyen las observaciones en el espacio `(vib, tmp)`.
- Aquí conviene explicar visualmente si las dos clases están bien separadas o si existe traslape.
- Es la mejor manera de introducir por qué la frontera de decisión no será perfecta.

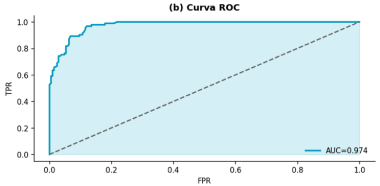
Referencia en el texto: ver **Figura 1**.



Gráfica 2 - Probabilidades y frontera de decisión

- Esta figura debe ilustrar cómo la regresión logística transforma la combinación lineal en probabilidad.
- También puede mostrar la frontera de decisión asociada al umbral de 0.5.
- Pedagógicamente, aquí se conecta la geometría del problema con la sigmoide.

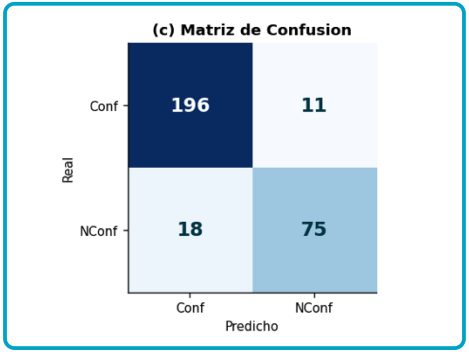
Referencia en el texto: ver **Figura 2**.



Gráfica 3 - Matriz de confusión o errores del clasificador

- Esta imagen debe ayudar a distinguir aciertos y errores del modelo.
- Es útil para conversar sobre **falsos positivos** y **falsos negativos**.
- Aquí se ancla la interpretación de precisión, recall y accuracy.

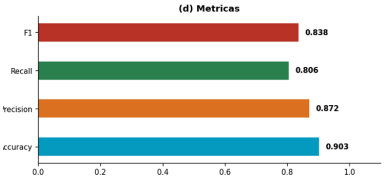
Referencia en el texto: ver **Figura 3**.



Gráfica 4 - Resumen final de métricas

- Esta figura debe resumir el desempeño global del modelo.
- Puede ser una barra comparativa, una tabla visual o un panel con *precision*, *recall*, *F1* y *accuracy*.
- Es ideal para cerrar la historia: qué tan bien funcionó el clasificador y qué se podría mejorar.

Referencia en el texto: ver **Figura 4**.



Conclusiones

- El notebook presenta una implementación compacta pero muy didáctica de regresión logística.
- La historia completa va de la simulación de datos a la evaluación del clasificador.
- La combinación de código claro, métricas estándar y visualización hace que sea muy útil para clase, workshop o presentación ejecutiva.

Próximo paso

Sustituye cada recuadro de Figura 1 a Figura 4 por las imágenes exportadas de tus gráficas y ajusta el texto de apoyo si quieres hacerlo todavía más específico al contenido exacto de cada visualización.